

# Java 刷题速成 — 总览与路线图

文档定位:C++ → Java 刷题迁移指南,以对照表 + 模板代码为主,跳过工程向理论。

用户画像:有 C++ 刷题基础(STL 50% 熟、OOP 会、Java 与 C++ 差异点 OK),目标速成。

平台:LeetCode + 牛客(ACM 模式)双覆盖。

## 一、开篇定调:三条原则

刷这一套文档,请你先记住这三条原则,后面所有内容都按这个思路来:

1. 对照优先:每讲一个 Java 概念,旁边必贴 C++ 写法。目的不是让你"学 Java",而是"翻译"你已有的 C++ 肌肉记忆。
2. 代码为王:每个 API 至少一段可直接抄的最小代码。文档里看不到完整可运行示例的 API,基本就不值得记。
3. 坑点先行:Java 刷题的 15 大经典坑全部前置到第 6 章。先看坑,再写代码,能省下 80% 的调试时间。

## 二、速成期望(先泼盆冷水)

重要:这一节决定你读这份文档的预期。

目标:

- 2-3 天看完即可上手 Java 刷题
- LeetCode 中等题通过率 70%+
- 牛客 ACM 模式 I/O 不再懵

不目标(明确告诉你这份文档不会覆盖什么):

- 不追求 100% 覆盖:只讲刷题会碰到的语法/API
- 不讲 Hard 题全通:难题仍需补算法,模板只解决"语言层"
- 不讲 Java 工程能力:Spring、反射、注解、多线程深入一律跳过
- 不讲算法原理:模板给到,原理点到为止

速成 ≠ 精通。这份文档帮你把"语言切换"的成本压到最低,刷题时不再因为语法卡壳。但遇到真正的算法难题(LeetCode Hard、Codeforces 紫名),还是得回去补算法基础。

完整"不做清单"见计划文件第八节,这里只列最常被误解的几条。

## 三、C++ → Java 总览对照表 (本节最高频)

2 分钟预备知识(看 00 时先知道这几点就够了,详细见 02/03):

- Java 泛型必须是对象:ArrayList<int> 不合法,要写 ArrayList<Integer>(Java 会自动把 int 装进 Integer,叫"装箱")
- String 是不可变对象:s += "x" 不是改原字符串,而是创建新对象;要"原地改"用 StringBuilder
- 方法签名 = 修饰符 + 返回类型 + 方法名(参数):public int[] twoSum(int[] nums, int target) 读作"公开方法,返回 int 数组,接收 int 数组和 int"
- 数组用 null 标记空位:Integer[] arr = {3, 9, null, null, 15, 7},这是 LeetCode 构造二叉树的标配写法

下面这张表是整份文档的总目录。做题时遇到 C++ 写法忘了 Java 怎么写,先查这张表定位到具体章节。

| C++ 知识点                         | Java 对应                                      | 见文档   |
|---------------------------------|----------------------------------------------|-------|
| cin / cout 输入输出                 | Scanner / BufferedReader + StreamTokenizer   | 01    |
| printf 格式化输出                    | System.out.printf / String.format            | 01    |
| vector<T>                       | ArrayList<T>(基本类型用 Integer 包装)               | 03    |
| list<T>                         | LinkedList<T>                                | 03    |
| stack<T>                        | ArrayDeque<T> 当栈用                            | 03    |
| queue<T>                        | ArrayDeque<T> / LinkedList                   | 03    |
| deque<T>                        | ArrayDeque<T>                                | 03    |
| priority_queue<T>               | PriorityQueue<T>(反的!默认小顶堆,C++ 是大顶堆)          | 03    |
| unordered_map<K,V>              | HashMap<K,V>                                 | 03    |
| unordered_set<T>                | HashSet<T>                                   | 03    |
| map<K,V>                        | TreeMap<K,V>                                 | 03    |
| set<T>                          | TreeSet<T>                                   | 03    |
| pair<T1,T2>                     | int[] / Map.Entry / 自定义类                     | 03    |
| sort                            | Arrays.sort / Collections.sort / List.sort   | 03/04 |
| string(可变)                      | String(不可变)+ StringBuilder(可变)               | 02/04 |
| ostringstream / sprintf(输出到字符串) | StringBuilder.append + toString              | 04    |
| to_string                       | String.valueOf / Integer.toString            | 04    |
| stoi / stoll                    | Integer.parseInt / Long.parseLong            | 04    |
| sqrt / pow                      | Math.sqrt / pow                              | 04    |
| accumulate(求和)                  | Stream.mapToInt(...).sum() / for 循环累加        | 04    |
| min_element / max_element       | Collections.min / max                        | 04    |
| binary_search(有序数组)             | Arrays.binarySearch                          | 04    |
| fill(数组填充)                      | Arrays.fill(arr, val)                        | 04    |
| next_permutation                | 无内建,需手写或转 List<Integer> 用 Collections        | 04/05 |
| lower_bound / upper_bound       | TreeMap.lowerKey / higherKey / 手写二分          | 03/05 |
| unique 去重                       | HashSet 重建(常用)                               | 04    |
| __builtin_popcount              | Integer.bitCount                             | 04    |
| bitset<128>                     | BitSet                                       | 04    |
| 引用 / 指针                         | 引用 + new,无 & 无 *                             | 02    |
| const                           | final                                        | 02    |
| static(文件作用域)                   | static(类共享)                                  | 02    |
| 异常处理                            | throws IOException / try-catch 最小化           | 01/02 |
| auto / 范围 for                   | for-each 增强 for                              | 02    |
| lambda                          | lambda + Comparator 链式                       | 02/04 |
| OOP 多态                          | 继承 + 接口(刷题少用)                                | 02    |
| 链表节点 struct ListNode            | 类 class ListNode { int val; ListNode next; } | 02    |



|       |                                                                  |    |
|-------|------------------------------------------------------------------|----|
| 二叉树节点 | 类 <code>class TreeNode { int val; TreeNode left, right; }</code> | 02 |
|-------|------------------------------------------------------------------|----|

怎么看这张表:

- 看到 C++ 写法 → 找到 Java 对应 → 跳到对应章节细看
- 比如看到 `priority_queue<int>` → Java 是 `PriorityQueue<Integer>`(注意小顶堆!) → 跳到 03 容器与 STL 对照

## 四、阅读路径(按你时间选)

路径 A:2-3 天速成(零基础 / 完全没碰过 Java)

00 → 01 → 02 → 03 → 04 → 05 → 06 → 07

每天安排建议:

- 第 1 天:00(本篇)+ 01 输入输出 + 02 语法差异。输入输出和语法先过关,不然连模板都跑不起来。
- 第 2 天:03 容器与 STL 对照 + 04 常用工具类。这是核心章节,容器不熟等于 Java 没法刷题。
- 第 3 天:05 刷题模式与模板 + 06 常见坑点速查 + 07 实战例题。直接上 7 道完整例题,从读到抄到改,基本就能上手了。

路径 B:1 天速成(有 STL 底子)

如果你的 STL 已经很熟,Java 容器 API 大部分能猜到,那就别从 01/02 慢慢磨了,直接走捷径:

00 → 03 → 05(只看 4 段骨架:ACM 模板 / 二分 / BFS-DFS / 双指针) → 速查表

具体操作:

- 30 分钟:00(本篇)通读,定方向
- 2-3 小时:03 容器与 STL 对照,重点看深讲条目(`ArrayList` / `ArrayDeque` / `HashMap` / `PriorityQueue` 四个)
- 2-3 小时:05 刷题模式与模板,先抄 ACM 模板 + 4 大算法骨架(排序/二分/BFS-DFS/双指针)
- 1 小时:速查表一页过,过完即可上手刷题
- 1 天路径默认你会写 `cin/cout`、会用 `vector/map/set/queue`,且能容忍 Java 语法上小卡壳(遇到再翻 02)。

牛客用户必看:1 天路径跳过了 01(输入输出),但 ACM 模式 I/O 跟 LeetCode 完全不同(要自己写 `main + throws IOException + BufferedReader`)。如果你主要刷牛客,务必把 01 第七节"ACM 模式完整骨架 v2"先抄一遍,再走 03 → 05。

>

坑点速扫必看:1 天路径跳过了 06(坑点),但前 3 个坑会直接编译失败或运行报错(`Stack` 别用 / `Integer` == 陷阱 / `throws IOException` 必加)。建议在速查表之前花 15 分钟把 06 至少扫一遍深讲容器相关的 6 个坑,先知道有坑。

路径 C:遇到不熟 API 时(任意时刻)

速查表.md → 定位到具体章节 → 看详解

不要从头翻文档。做题卡住 → 翻速查表查 API 名称 → 跳到对应章节看完整示例 → 回到题目继续。



## 五、8 篇文档一句话摘要

| #  | 文档           | 一句话                                              |
|----|--------------|--------------------------------------------------|
| 00 | 总览与速成路线      | 本文档,定方向                                          |
| 01 | 输入输出篇        | Scanner / BufferedReader / PrintWriter + ACM 模板  |
| 02 | 语法差异速览       | C++ 转 Java 必看的 10 个纯语法点                          |
| 03 | 容器与 STL 对照 ★ | 13 个容器对照,核心章节                                    |
| 04 | 常用工具类        | Arrays / Collections / Math / String + Java 8 必备 |
| 05 | 刷题模式与模板      | ACM + LeetCode 模式 + 10 大算法模板                     |
| 06 | 常见坑点速查       | 15 大经典坑,救命文档                                     |
| 07 | 实战例题         | 7 道题完整流程(输入→思路→代码→对照)                            |

标注的两篇(03 容器、06 坑点)是你刷题时反复回头翻的,其他 6 篇读一遍掌握结构即可。

## 六、速查表使用指南

Java刷题速成/速查表.md 是单页 CheatSheet,定位和用法如下:

用法 1:做题中遇到不熟 API(主用法)

- 做题时,某个 API 忘了怎么写(比如 HashMap.getOrDefault 第二个参数是默认值还是 key?)
- 翻速查表查 API 名称 → 看到对应章节号 → 跳过去看完整代码
- 不要从 00 一路翻到 07,速度太慢

用法 2:读完全部 8 篇后(考前总复习)

- 速查表设计成"一页过",关键 API 全部一行一条
- 考前一晚快速过一遍,比临时翻文档快 10 倍

速查表何时生成?

速查表在阶段二末尾(随 05 / 06 / 07 写完)生成,所以现在还看不到。生成后会放在:

Java刷题速成/速查表.md

写完时会在文档索引里更新,届时 8 篇正文 + 1 张速查表形成完整闭环。

## 七、章节依赖关系(谁依赖谁)

为了避免读到一半发现"这一节要看的 API 还没讲",先放一张依赖图:



00(本篇)  
↓  
01(输入输出)← 不依赖任何章节  
↓  
02(语法差异)← 引用 01 的 throws IOException 概念  
↓  
03(容器对照)← 引用 02 的 final / static / 类与对象  
↓  
04(工具类)← 引用 03 的容器 API + 02 的 lambda  
↓  
05(刷题模板)← 引用 01 模板 + 03 容器 + 04 工具  
↓  
06(坑点速查)← 引用 01-05 全部  
↓  
07(实战例题)← 引用 01-06 全部  
↓  
速查表(汇总 01-07)

简言之:从前往后读,别跳。跳着读很容易遇到"这里说的 API 还没讲"的问题。

## 八、风格约定(读前必看)

- 代码块:Java 高亮为主,小段 C++ 用 cpp 高亮作为对照
- 符号: 标记高频必会;不用 等 Unicode(为未来 PDF 兼容)
- 错误/正确:用 错误 / 正确 标记代码示例,不用 错误: 正确: 标点(避免视觉混淆)
- API 旁注:关键 API 旁注中文释义,比如 peek() — 查栈顶
- 章节末尾:每篇都留"练习建议"块,指向 LeetCode / 牛客具体题目

## 练习建议

- 完成 00 阅读后,花 10 分钟浏览 03 的对照表(不细看,只看结构,有个印象)
- 然后从 01 开始按顺序读
- 如果时间紧,直接走 1 天路径:00 → 03 → 05(骨架部分)→ 速查表
- 读完 8 篇后,速查表作为长期工具,做题时反复翻

## 下一步

打开 01\_输入输出篇.md,从 ACM 模板开始。

